LLM Agent Planning in Text-Based Environments with Deceptive NPCs

Hritik Bharucha
Computer Science and Engineering
University of California, San Diego
hbharucha@ucsd.edu

Kuber Shahi
Computer Science and Engineering
University of California, San Diego
kshahi@ucsd.edu

Akshay Ghosh
Computer Science and Engineering
University of California, San Diego
akghosh@ucsd.edu

Basar Demir
Computer Science and Engineering
University of California, San Diego
bdemir@ucsd.edu

Rohan Arcot
Computer Science and Engineering
University of California, San Diego
rarcot@ucsd.edu

Abstract

Large language model (LLM) agents are increasingly deployed in interactive environments where they must reason, plan, and act based on information from external entities. However, most existing evaluation benchmarks assume cooperative environments where all information sources are truthful. In real-world settings, agents must often reason under deception and uncertainty. We introduce a text-based planning environment where LLM agents must complete long-horizon tasks while interacting with non-player characters (NPCs) that may provide misleading or deceptive information. The task requires agents to collect sigils across various locations and unlock a vault in the correct order, all while operating under a strict action budget. We evaluate four agent designs, varying in trust modeling, memory, and reflection, against multiple deception levels and environment complexities. We find that explicit trust modeling is crucial for robustness: agents that maintain dynamic trust scores achieve perfect task success even when most NPCs are deceptive, while naive agents exhibit cliff-edge failure once deception becomes common. Surprisingly, adding reflective reasoning degrades performance due to behavioral over-caution and inefficient planning. A cross-model comparison between GPT-OSS-120B and Llama-4-Scout, combined with a hint ablation study, reveals that planning capability, not deception reasoning, is the primary bottleneck for weaker models. Our results suggest that reliable agent planning in deceptive environments depends more on structured trust representations than on simply increasing reasoning depth.

1 Introduction

Large language models (LLMs) have demonstrated impressive reasoning and planning capabilities when deployed as autonomous agents. Recent work explores agents capable of interacting with environments, querying tools, and coordinating actions to achieve complex goals. However, most

Preprint.

existing evaluation environments assume cooperative conditions in which all information provided to the agent is truthful and consistent.

In many real-world scenarios, this assumption does not hold. Agents operating in open environments may receive unreliable, contradictory, or adversarial information. Early incorrect beliefs can propagate through the agent's internal reasoning state, leading to cascading planning failures. Understanding how LLM agents handle misinformation is, therefore, critical for robust agent design.

In this work, we study *planning under deception*. We introduce a controlled text-based environment where an LLM agent must complete a long-horizon task while interacting with NPCs that may intentionally provide misleading information. The agent must collect sigils scattered across locations and unlock a vault in the correct order. NPCs provide clues about sigil locations and vault ordering, but some NPCs lie depending on the agent's trust level. This setting allows us to study a key question:

> *Can LLM agents learn who to trust, and does more reasoning actually help when information sources may be deceptive?*

We evaluate four agent designs that differ in how they handle trust, memory, and reasoning. Our results reveal four key findings. First, explicit trust modeling dramatically improves robustness: agents that track trust scores for NPCs maintain perfect success even when most NPCs are deceptive. Second, naive agents exhibit a *cliff-edge failure* where performance remains high under low deception but collapses once deceptive information becomes common. Third, reflection-based reasoning hurts performance by introducing excessive caution and inefficient action selection. Fourth, a hint ablation study reveals that *planning capability, not deception reasoning*, is the primary bottleneck for weaker models: Llama-4-Scout jumps from 0% to 93% on Belief-Tracking when structured payload hints are provided, though trace analysis shows this success comes from consistently querying a truthful NPC rather than genuine trust reasoning. Together, these findings suggest that robustness in agentic environments depends more on structured trust calibration than on simply increasing reasoning depth.

2 Related Work

LLM Agents in Interactive Environments. Recent work has explored LLM-based agents capable of interacting with environments, performing multi-step reasoning, and executing plans. ReAct [Yao et al., 2022] interleaves reasoning traces with actions, enabling agents to dynamically revise plans in text-based and web environments. Reflexion [Shinn et al., 2023] extends this paradigm by adding verbal self-reflection over past failures, stored in an episodic memory buffer, to improve decision-making across subsequent trials. Voyager [Wang et al., 2023] demonstrates open-ended skill acquisition in Minecraft through an LLM-driven curriculum and a growing code library. Generative Agents [Park et al., 2023] equip LLM-based characters with memory, reflection, and planning to produce believable social behavior in a simulated town. Chain-of-thought prompting [Wei et al., 2022] has become a standard technique for eliciting step-by-step reasoning in LLMs. While these approaches improve reasoning and planning in cooperative environments, they rarely consider settings where information sources may be adversarial or deceptive.

Deception and Social Deduction Benchmarks. Another line of work studies deception in social-deduction and hidden-role environments. Recent benchmarks use games such as *Werewolf*, *The Traitors*, and *Secret Hitler* to evaluate LLM behavior under asymmetric information, persuasion, and hidden-role inference [Bailis et al., 2024, Curvo, 2025, Izora and Teuscher, 2025]. A recurring finding is a deception asymmetry: models can often generate convincing deception more effectively than they can identify deception in others [Curvo, 2025, Agarwal et al., 2025]. The MACHIAVELLI benchmark [Pan et al., 2023] evaluates agents in 134 text-based Choose-Your-Own-Adventure games, finding that agents trained to maximize reward tend to adopt deceptive and power-seeking behaviors. These settings are useful for studying bluffing, lie generation, and suspicion, but they typically measure trust through social signals such as suspicion scores or voting behavior rather than grounded task completion [Agarwal et al., 2025]. Our setting instead treats deception as a planning problem: false NPC statements redirect exploration, waste actions, and prevent successful task completion within a fixed budget.

Trust and Information Reliability. Trust modeling has been studied extensively in multi-agent systems and human-computer interaction. Several approaches use suspicion scores or reputation systems to track agent reliability over time. However, these mechanisms have rarely been integrated

into LLM agent architectures that perform sequential planning. Generative Agents [Park et al., 2023] maintain memory and reflection but assume cooperative social dynamics. Reflexion [Shinn et al., 2023] reflects on failures but does not model the reliability of external information sources. Our Belief-Tracking agent explicitly maintains per-source trust scores that are updated based on grounded environmental feedback (successful or failed searches), providing a structured alternative to unconstrained verbal reflection.

Our work differs from prior research in two key ways. First, we focus on *task-oriented deception*, where misinformation leads to failed plans and wasted actions rather than social penalties or reduced reward. Second, we systematically compare how different agent scaffolds (trust scores, memory, reflection) handle deceptive information in long-horizon planning environments with strict resource constraints.

3 Environment and Agent Design

3.1 Task Environment

We design a text-based planning environment in which an LLM agent must collect sigils from different locations and unlock an ancient vault. The agent begins at a central location (`village_square`) in a connected world and must complete the task within a limited step budget. At each turn the agent selects one of four actions: `move` (travel to an adjacent location), `talk` (query an NPC about a sigil location or vault ordering), `search` (search the current location for a sigil), or `unlock` (attempt to open the vault with collected sigils in a specified order). NPCs distributed across the map provide clues, but some are deceptive. The agent must determine which information sources are reliable while completing the task within its action budget.

We use two environment configurations (Table 1). The *default world* uses a hub-and-spoke topology with 5 locations (village square connected to forest shrine, cave pool, river dock, and the vault), 3 sigils, and an 18-step budget (optimal: 15 steps). The *extended world* uses a branched topology with 7 locations (two branches from the village square, with the vault accessible only through a chain of intermediate locations), 4 sigils, and a 25-step budget (optimal: 19 steps). The branched topology increases the cost of wrong turns because the agent cannot return directly to the hub but must backtrack through intermediate nodes.

Table 1: Environment configurations.

World	Locations	Sigils	Budget	Optimal	Topology
Default	5	3	18	15	Hub-and-spoke
Extended	7	4	25	19	Branched

3.2 NPC Deception

Each NPC is assigned either a truthful or deceptive role. Truthful NPCs always provide correct information. Deceptive NPCs lie when the agent's trust in them is high, implementing adaptive deception that targets trusting agents and reverts to truth-telling when trust is low to rebuild credibility. We control the difficulty of each episode by varying the *liar ratio*, the fraction of NPCs assigned a deceptive role, from 0.0 to 0.7.

3.3 Agent Variants

We evaluate four LLM agent designs (Table 2). The **Naive** agent accepts NPC statements at face value and acts on the most recent clue, with high initial trust and minimal trust decay even after failures. The **Belief-Tracking** agent maintains a dynamic trust score $T \in [0, 1]$ for each NPC, initialized at 0.5, and weights incoming information by these scores. Trust increases when an NPC's claim is confirmed by observation and decays when contradicted; once trust falls below a threshold, the agent distrusts that NPC. The **Reflection-Enhanced** agent extends belief-tracking with periodic reflective reasoning, in which a separate LLM call analyzes past interactions to identify suspicious NPCs, contradiction patterns, and suggested next actions; this reflection output is injected into the agent's

payload for subsequent action selection. The **Memory + Trust** agent tracks all past NPC statements in a structured memory, detects contradictions between claims, and combines this with explicit trust scores, but omits the reflection layer. It can prefer majority-supported claims when multiple NPCs disagree while also weighting information by trust.

Table 2: Agent variants and their key mechanisms.

Agent	Trust Scores	Memory	Reflection
Naive			
Belief-Tracking	✓		
Reflection-Enhanced	✓		✓
Memory + Trust	✓	✓	

3.4 Prompt Design

Each agent variant is driven by a system prompt that specifies the world graph, the action space, the required output format (structured JSON), and variant-specific reasoning instructions. All agents share a common preamble that is dynamically generated from the world configuration:

```
You are the primary planning agent in a text-based strategic game.
GAME RULES: You start at village_square.  The world has 5 locations:
[connections listed].  Your goal:  collect 3 sigils (sun_sigil,
moon_sigil, river_sigil) from various locations, then go to
ancient_vault and unlock it with the correct sigil order.  WARNING:
Some NPCs may be DECEPTIVE. You do NOT know which ones.
AVAILABLE ACTIONS: [JSON schema for talk, move, search, unlock]
```

The preamble also includes an example optimal action sequence showing a talk-move-search-move-unlock pattern, followed by variant-specific strategy instructions. For example, the Belief-Tracking agent receives:

```
STRATEGY: You maintain a trust score T in [0,1] for each NPC. Use
the trust_scores provided in the payload to weight NPC claims.
Prefer claims from high-trust NPCs.  If an NPC's advice led to a
failed search or failed vault attempt, their trust should drop.
If an NPC's advice led to a successful find, their trust should
increase.  When claims conflict, go with the higher-trust source.
Treat NPC information as noisy evidence, not fact.
```

The Naive agent receives: "`STRATEGY: You trust all NPCs equally. Accept information at face value. Act on the first information you receive.`" The Reflection-Enhanced agent is additionally instructed to use a `reflection` field in the payload for guidance on suspicious NPCs. The Memory+Trust agent is told to prefer majority-supported claims when NPCs disagree.

Reflection is handled by a separate system prompt and LLM call that runs inside the agent's action-selection loop (before the environment step counter increments, so it does not consume a step):

```
You are reflecting on your performance in a strategic text game with
deceptive NPCs.  Given your memory summary, trust scores, and recent
failures, produce a reflection.  TASKS: 1.  Identify which NPCs are
likely deceptive based on patterns of failed advice.  2.  Note any
contradictions that suggest specific NPCs are lying.  3.  Suggest
the best next focus.
```

At each turn, the agent receives a rich payload containing its current location, inventory, collected sigils, the full structured memory (all past NPC claims organized by fact, detected contradictions, confirmed environment facts), current trust scores, and the list of previously asked NPC-topic pairs to avoid redundant queries.

In the primary evaluation setting, NPC responses are generated deterministically from policy rules rather than by an LLM, isolating the agent's reasoning from NPC generation quality.

3.5 Evaluation Metrics

We report two primary metrics: **Task Success Rate** (whether the agent collected all sigils and unlocked the vault correctly within the budget) and **Average Steps** (actions per episode, measuring planning efficiency).

4 Experimental Results

To contextualize performance, we include two baselines: an Oracle agent that computes the optimal path via breadth-first search on the ground-truth state (upper bound), and a Random agent that selects uniformly among valid actions (lower bound). The oracle achieves perfect success at exactly the optimal step count; the random agent achieves 0%. We evaluate across deception levels using two LLM backends: GPT-OSS-120B and Llama-4-Scout, both accessed via the UCSD TritonAI API.

4.1 Default World Performance

Figure 1 shows task success rates in the default world (5 runs per setting, 150 total episodes). Trust-based agents (Belief-Tracking and Memory+Trust) achieve perfect success at every deception level, matching the oracle even when 70% of NPCs are deceptive. The Naive agent exhibits a cliff-edge failure: 100% success at liar ratios 0.0–0.3, then collapses to 0% at 0.5. Trace analysis reveals this is driven by NPC role assignment: at a liar ratio of 0.3, the agent's preferred NPC is truthful, but at 0.5 it flips to deceptive, sending the agent to the wrong locations. Each wrong referral wastes 2–4 steps, and the 18-step budget leaves no room for recovery. The Naive agent maintains high trust even after failed searches, so it cannot self-correct. Reflection achieves only 28% overall, performing worse than Naive and failing even at a liar ratio of 0.0, where all NPCs are truthful. The trust-based vs. Reflection comparison is statistically significant under Wilson score 95% confidence intervals (n=25 per variant).

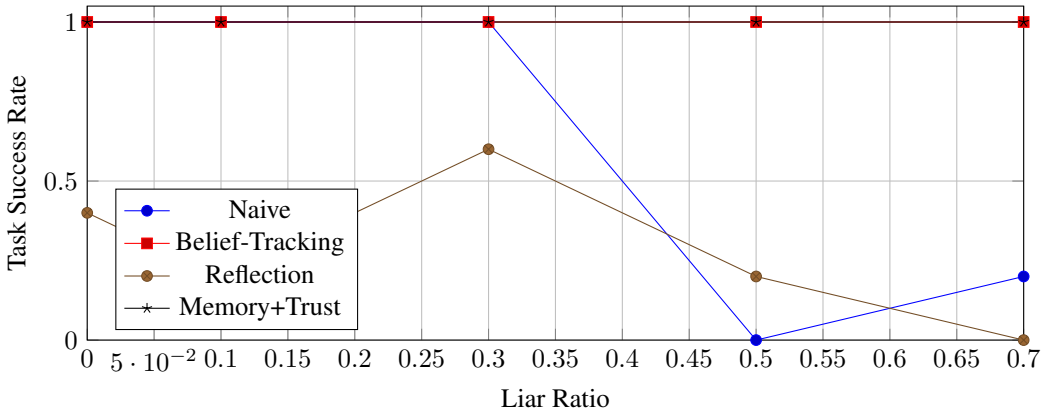

Figure 1: Success rate vs. deception level in the default world. Trust-based agents remain robust at all levels; the Naive agent collapses sharply at a liar ratio of 0.5.

4.2 Step Efficiency

Trust-based agents remain close to the optimal path length, completing tasks in approximately 15–16 steps (Figure 2). Belief-Tracking averages 15.4–16.4 steps across deception levels; Memory+Trust is comparable at 15.6–16.4. Naive and Reflection agents frequently exhaust the full 18-step budget under deception. These failures occur because incorrect information triggers unnecessary travel: each wrong referral costs 2–4 actions (move out, failed search, move back), and the budget leaves only 3 spare steps above the optimal path.

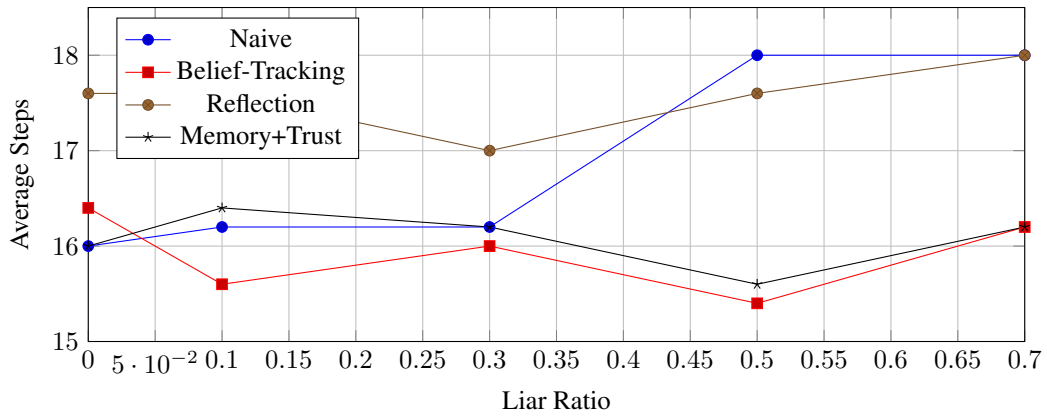

Figure 2: Average steps per episode in the default world. Trust-based agents stay near optimal (15 steps); Reflection consistently hits 17–18 even at a liar ratio 0.0.

4.3 Extended World Results

Figure 3 shows results in the extended world (3 runs per setting, 72 episodes). The Naive agent degrades faster than in the default world: 33% at a liar ratio of 0.3, 0% at 0.5 and above (overall: 33%, down from 64%). The branched topology means recovery requires backtracking through intermediate nodes rather than a single hub return. Trust-based agents remain at 100%. Reflection improves to 58% overall (vs. 28% in the default world), likely because the larger budget (25 vs. 18 steps) absorbs more of the overhead, though it remains variable (100% at a liar ratio of 0.3 but 33% at 0.5). Belief-Tracking achieves near-optimal 19.0–19.3 steps; Memory+Trust is close at 19.3–20.0.

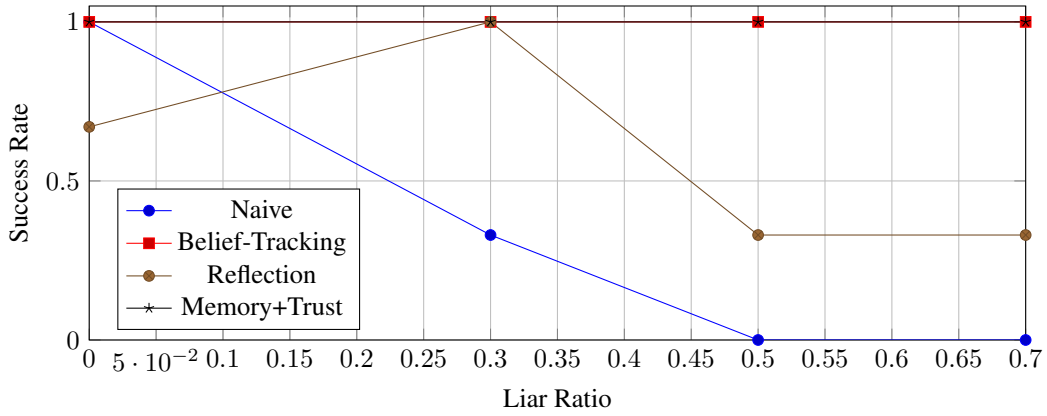

Figure 3: Extended world performance. Naive degrades from 64% to 33% overall compared to the default world; trust-based agents remain at 100%.

4.4 Cross-Model Comparison

Table 3 compares GPT-OSS-120B and Llama-4-Scout without hints (3 runs per setting, 60 episodes per model). GPT-OSS-120B succeeds with trust-based strategies; Llama-4-Scout fails across all variants. Trace analysis of Llama's failures reveals three modes: (1) at locations with NPCs, Llama prioritizes talking over searching and never collects the sigil; (2) Llama attempts moves between non-adjacent outer locations (e.g., `cave_pool` → `river_dock`) that do not exist in the hub-and-spoke graph; (3) with belief-tracking's lower initial trust, Llama repeatedly asks the same NPCs for confirmation instead of acting, burning all 18 steps on talk actions. Importantly, Llama produces valid JSON, so this is a planning failure, not a formatting failure.

Table 3: Cross-model comparison: overall success rates (no hints, default world).

Model	Naive	Belief-Tr.	Reflect.	Mem+Trust	Overall
GPT-OSS-120B	0.64	1.00	0.28	1.00	0.73
Llama-4-Scout	0.00	0.00	0.00	0.00	0.00

4.5 Hint Ablation: Diagnosing Planning vs. Reasoning Bottlenecks

To disentangle whether weaker models fail at *trust reasoning* or at *action-sequence planning*, we introduce structured payload hints. When the agent is at a location where a needed sigil is believed to be, it receives: ‘‘`You MUST use ‘search’ action NOW. Do NOT talk to NPCs here.`’’ When it has gathered enough claims but remains at the hub, it receives: ‘‘`STOP talking and use ‘move’ to go to [location] NOW.`’’ Additional prompt rules instruct the agent not to re-ask about topics already queried and to prioritize efficiency. These hints reduce the planning burden while leaving the trust-reasoning challenge intact. Table 4 shows the results.

Table 4: Hint ablation: overall success rates with structured payload hints.

Model	Naive	Belief-Tr.	Reflect.	Mem+Trust	Overall
GPT-OSS-120B	0.95	1.00	1.00	1.00	0.99
Llama-4-Scout	0.00	0.93	0.93	0.07	0.48

For GPT-OSS-120B, hints are redundant for Belief-Tracking and Memory+Trust (already at 100%), but rescue Reflection from 28% to 100% and Naive from 64% to 95%. This confirms that reflection's cost is primarily in planning efficiency: the hints override the over-cautious behavior induced by the reflection payload.

For Llama-4-Scout, hints rescue Belief-Tracking and Reflection from 0% to 93% but fail to help Naive (still 0%) and barely help Memory+Trust (0% $\rightarrow$ 7%). However, trace analysis reveals a critical confound: across all 15 successful Belief-Tracking+hints episodes, Llama makes 64 talk actions, all directed at a single NPC that is always assigned the truthful role in the current roster. Trust scores for all other NPCs remain at 0.50 (the prior), meaning Llama never interacts with deceptive NPCs. Llama's 93% is therefore not evidence of trust reasoning; it sidesteps the deception problem by latching onto a single always-truthful NPC. The hints provide the planning scaffold, and this NPC provides guaranteed-correct information. Naive remains at 0% because no planning assistance helps when the agent has no mechanism to discount deceptive information.

5 Discussion

Our results highlight the importance of explicit trust calibration in agent design. Agents that maintain structured representations of information reliability remain robust even when most sources are deceptive. Agents that accept information at face value are highly vulnerable: early incorrect beliefs propagate through the planning process and lead to cascading failures.

Although Belief-Tracking and Memory+Trust achieve identical success rates, their strategies differ. Belief-Tracking commits to an information source early and acts quickly (exploitative strategy), sometimes achieving 15-step completion. Memory+Trust queries multiple NPCs before committing (exploratory strategy), incurring 1–2 extra steps but gathering redundant evidence. Despite these behavioral differences, trace analysis at a liar ratio of 0.7 reveals that both agents sometimes assign the highest trust to a deceptive NPC whose location claims happen to be correct (the NPC lies about the vault order instead). This indicates that task success does not require accurate deception detection and that the current environment may not fully stress-test trust calibration.

5.1 Why Reflection Hurts Performance

Reflection-Enhanced achieves only 28% overall, worse than Naive (64%), and fails even at a liar ratio of 0.0. Crucially, reflection does *not* consume a step: it runs inside the agent's action-selection function before the environment step counter increments. The performance drop comes from how

the LLM changes its behavior after reading the reflection payload. First, the reflection module flags NPCs as suspicious even when evidence is ambiguous, causing the agent to re-query NPCs it has already consulted (over-suspicion). Second, after reading a reflection that says "some NPCs may be deceptive," the agent cycles between re-consulting and re-reflecting instead of acting on available information (analysis paralysis). Third, the reflection content itself is often reasonable, but injecting it into the prompt causes the LLM to second-guess correct information even at a liar ratio of 0.0 (behavioral contamination).

The hint ablation confirms this interpretation: with hints, Reflection jumps from 28% to 100% on GPT-OSS-120B, because the hints override the over-cautious behavior. In the extended world (25-step budget), Reflection improves to 58%, consistent with the hypothesis that its cost is budget-relative. This suggests that reflection-style architectures should be evaluated under resource constraints, not just in unlimited-step settings.

5.2 Planning as the Primary Bottleneck

The cross-model comparison and hint ablation together reveal that planning capability, not deception reasoning, is the primary bottleneck for weaker models. The hint gap (success with hints minus success without) measures how much failure is attributable to action-selection planning: GPT-OSS Belief-Tracking has a 0% gap (autonomous planning succeeds), GPT-OSS Reflection has a 72% gap (most failures are planning overhead), Llama Belief-Tracking has a 93% gap (nearly all failure is planning), and Llama Naive has a 0% gap (failure is fundamental, not planning-related).

However, Llama's hint success should be interpreted cautiously. Because the NPC role assignment is fixed (one NPC is always truthful) and Llama exclusively queries that NPC, its success demonstrates hint-following with a favorable role assignment rather than genuine deception reasoning. A stronger test would randomize which NPC is truthful across episodes. Nonetheless, the finding has practical implications: improving the planning scaffold may yield larger gains than improving the reasoning scaffold for models below the frontier of planning capability.

6 Limitations

This study has several important limitations. Our setup uses deterministic mock NPCs; this isolates agent planning but does not capture the messiness of real generative deception. Deceptive NPCs in our environment lie primarily about vault ordering, not sigil locations, so agents can succeed without correctly identifying liars. A harder variant should require agents to directly confront conflicting location advice. Per-cell sample sizes are small (5 runs in the default world, 3 in extended/hint experiments), so percentages should be interpreted as directional. NPC role assignment is not fully randomized (one NPC is always truthful), creating a confound for Llama's hint results. We evaluate only two LLM backends; broader model coverage would strengthen generalizability.

Future work could explore richer environments where deception directly affects location advice, adversarial NPC adaptation, and fully randomized NPC roles. The hint ablation methodology could be extended to provide finer-grained planning-vs-reasoning decompositions across a wider range of models.

7 Conclusion

We studied LLM agent planning under deception in text-based environments. Belief-Tracking and Memory+Trust agents achieve perfect robustness across both environment sizes, while Naive trust breaks sharply at a deception threshold, and Reflection is harmful under tight budgets. A cross-model hint ablation decomposes failures into planning and reasoning components, revealing that planning capability is the primary bottleneck for weaker models; however, trace analysis shows that current success metrics can overstate trust reasoning when the task permits agents to avoid rather than solve the deception problem. The main takeaway is that structured trust helps LLM agents plan under misinformation, while truly reliable deception understanding remains an open problem. Our codebase and all results are publicly available.[1]

[1] `https://github.com/kubershahi/llm-agent-deception`

References

Mrinal Agarwal, Saad Rana, Theo Sundoro, Hermela Berhe, Spencer Kim, Vasu Sharma, Sean O'Brien, and Kevin Zhu. Wolf: Werewolf-based observations for llm deception and falsehoods. *arXiv preprint arXiv:2512.09187*, 2025. URL `https://arxiv.org/abs/2512.09187`.

Suma Bailis, Jane Friedhoff, and Feiyang Chen. Werewolf arena: A case study in llm evaluation via social deduction. *arXiv preprint arXiv:2407.13943*, 2024. URL `https://arxiv.org/abs/2407.13943`.

Pedro M. P. Curvo. The traitors: Deception and trust in multi-agent language model simulations. *arXiv preprint arXiv:2505.12923*, 2025. URL `https://arxiv.org/abs/2505.12923`.

Kaj Hansteen Izora and Christof Teuscher. Exploring the potential of large language models (llms) to simulate social group dynamics: A case study using the board game "secret hitler". *Northeast Journal of Complex Systems*, 7(2):Article 5, 2025. doi: 10.63562/2577-8439.1111. URL `https://orb.binghamton.edu/nejcs/vol7/iss2/5`.

Alexander Pan, Jun Shern Chan, Andy Zou, Nathaniel Li, Steven Basart, Thomas Woodside, Hanlin Zhang, Scott Emmons, and Dan Hendrycks. Do the rewards justify the means? measuring trade-offs between rewards and ethical behavior in the machiavelli benchmark. In *International conference on machine learning*, pages 26837–26867. PMLR, 2023.

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.